

Block-Diffusion OD Planning — 통합 결과 (v3)

상태 (2026-07-20, v3). 본 문서의 모든 수치는 v2 백본(시드된 80/20 split로 재학습한 14개 모델: mask×7, graph d=2.0×7) 기준으로 재측정되었다. 평가: normal 과제는 학습에 쓰이지 않은 test split에서 추출한 공유 held-out 1,000쌍(\$2는 생성 시드 3개 mean±std), 예외 과제는 discriminator 학습에서 제외된 except_0 예약 1,000쌍. 발전 과정과 전체 ablation 서사는 RESULTS_BD_v1.pdf (아카이브)에, 수식 전문은 BD_GUIDANCE_FORMULATION.pdf · BD_CODE_VERIFICATION_K0/EN.pdf · BD_ARCHITECTURE_K0.pdf 에 있다. v1 체크포인트·disc는 v1_* 로 아카이브.

1. 설정과 아키텍처

BD3-LM식 block diffusion (블록 간 semi-AR, 블록 내 이산 diffusion), 6층/dim 256 DiT(adaLN), porto.v3.normal (정점 1,390, 경로 193만), 1 epoch = 48.2k iter (bs 32, lr 5e-4). 캔버스 [dst, ori, v..., v_L=dst, END...] : END tail은 dst 블록 내 학습 대상, PAD 블록 제외. 커널: mask(absorbing, SUBS, first-hitting — 데이터 최대 길이 59라 blk64는 사실상 MDLM)와 graph(uniform-state CTMC, d=2.0, T=100). 생성은 open-ended (dst→hit / END→miss).

1.1 상세 설정

디노이저	DiT 6층 / dim 256 / heads 8 / cond 128 / dropout 0.1, adaLN(zero-init), 학습형 절대 위치(양 절반 공유), node2vec(100d) 임베딩 초기화
학습	1 epoch = 48.2k iter, bs 32, Adam lr 5e-4, grad clip 1.0, 조건 dropout 0.1(O·D 동시 null), mask: t~U[10 ⁻³ ,1] 블록별 / graph: T=100, β∈[10 ⁻⁴ ,10], d=2.0
데이터/분리	porto v3-0.05 normal: 정점 1,390, 최단경로 193만 (최대 길이 59). 순서 고정(shuffle=False)+torch seed 1의 80/20 random_split → 멤버십 완전 재현. 평가: test split에서 seed 777로 추출한 공유 1,000쌍
예외 데이터	v4-0.05 except_0...99 (edge 5% 제거; normal 그래프는 v3와 동일). disc 학습 = 예약 1,000행 제외 후 1% (19.3k 경로); 평가 = 예약 1,000쌍 (disc가 못 본 데이터)
Discriminator (IW)	부분 경로 입력(클린, 랜덤 절단), 2층/dim128 transformer + GraphSAGE(2층) + 전이 피쳐[edge-existence, is-transition, deg-ratio] + masked mean pooling + bounded logit λ=4, BCE(label smoothing 0.05), Adam 1e-3, 4,000 스텝, bs 128. negative = 해당 블록 v2 모델의 unconditional 생성 20k
D-CBG 분류기	동일 아키텍처(+MASK 토큰, t-조건 임베딩), 입력 = 생성 시점 corruption을 입힌 노이즈 상태 (그들 프로토콜), 같은 1% 데이터.같은 negative pool(그들 방식으로 노이즈)
Guidance	N=100 mean-field 후보/reveal, w=exp(bounded logit)=D/(1-D), 매 reveal 적용, self-normalized, ESS 로깅; adj-proposal: 공개-이웃 기준 A_scn 마스크 (Lemma 3); D-CBG: γ=4 (γ≤2 무력)
하드웨어	RTX 3090 (24GB) ×4, torch 1.12.1+cu113

1.2 성능 지표 정의

hit	후처리 이전 모델 출력이 dst에 스스로 도달한 비율. 항상 pre-refine으로 측정되어 후처리로 부풀 수 없음 (raw에서는 hit = arrival)
arrival	(해당 변형의 후처리까지 적용한) 최종 경로의 마지막 정점이 dst와 일치하는 비율
valid	경로의 모든 연속 정점쌍이 평가 그래프의 edge인 비율 (경로 단위 all-or-nothing; 예외 평가에서는 A _{except0} 기준) — edge 오류율이 낮아도 경로 길이만큼 증폭됨 (§2.3)
valid∧hit (raw)	valid이면서 hit인 비율 — 후처리와 무관한 "사용 가능 경로율". P1+P3 하에서는 raw hit과 일치(유일하게 남는 모델 의존량)
EM / PC	저장소 표준 채점(eval_shortest.evaluate_em_pc): EM = 생성 경로가 해당 OD의 참조 최단경로와 정확 일치하는 비율, PC = 참조 경로 정점의 부분 커버리지(겹침) 점수. 둘 다 경로별 계산 후 평균
DTW	생성/참조 경로의 (위도,경도) 시퀀스 간 dynamic time warping 거리(L1, ~km 스케일)를 max(길이)로 정규화한 평균 — 기하학적 궤적 유사도
len err / len corr	생성 길이 - 실제 길이 의 평균 / 두 길이의 Pearson 상관 — 창발 길이의 보정도 (§2.3: hit 경로에서는 dst-절단 기하가 지배)
inv-edge / rem-edge / inv-node	invalid edge 비율(edge 단위) / 그중 "제거된 edge"(normal에는 있으나 시나리오에서 폐쇄) 사용 비율 / invalid edge에 접한 정점 비율
ESS	reveal당 importance weight의 유효 표본 수 (Σw) ² /Σw ² ∈ [1, N]. N≈100이면 가중치 균등(guidance 무력), 1 근처면 분산 폭발 — 두 실패 모드를 구분하는 계기판
patch	후처리가 삽입한 평균 토큰 수 (P1 브리지 + P3 graft)
wall-clock (s/path)	경로 1개를 생성하는 데 걸린 실측 벽시계 시간: 유티 RTX 3090 1장, 배치 100, 200~1,000쌍 생성 루프(모델 forward + guidance 연산 포함, 데이터 로드·채점 제외)의 경과시간 ÷ 경로 수. 분류기 "호출 수"와 달리 배치 효율·backward 비용·시퀀스 길이까지 반영된 실제 속도 — \$5에서 D-CBG vs IW 비교에만 보고

그림 1. Block-diffusion 동작 원리 — 특히 이전 블록 정보의 유입 경로 (코드 세부과 라인 번호: pdfs/BD_ARCHITECTURE_K0.pdf)

A. 캔버스와 길이 스택 (blk4 예시: build_canvas)

dst ori V1 V2 V3 V4 V5 V6=dst END END PAD PAD

노랑 = 클린 조건 prefix (loss 제외); END tail은 dst 블록 내부에서 학습 대상; PAD 블록은 loss 제외.

B. 이전 블록 정보의 유입 = self-attention 단일 경로

	B1	B2	B3	B1'	B2'	B3'
B1	BD	.	.	.	.	.
B2	.	BD	.	OBC	.	.
B3	.	.	BD	OBC	OBC	.
B1'	.	.	.	BD	.	.
B2'	.	.	.	BC	BD	.
B3'	.	.	.	BC	BC	BD

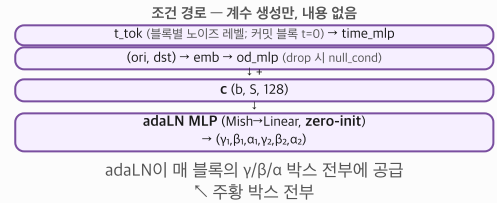
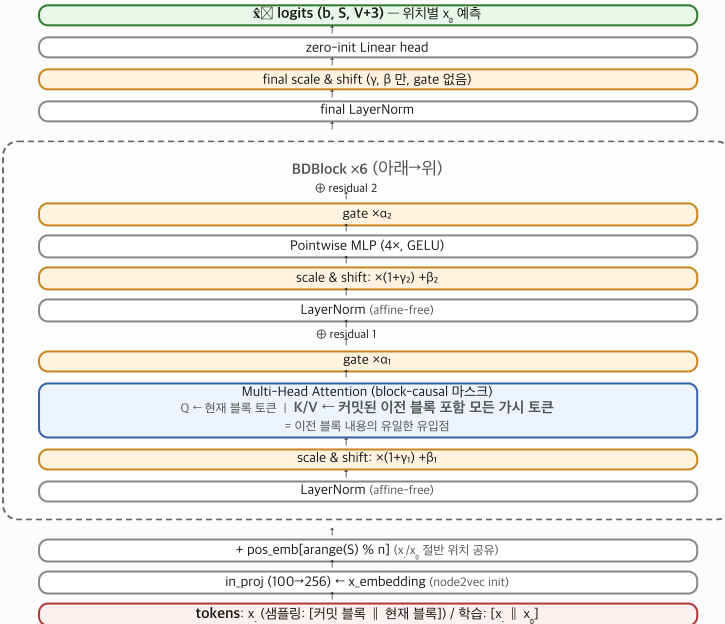
	B1	B2	B3*
B1		✓	.
B2		✓	✓
B3*		✓	✓

왼쪽 (학습, get_block_diff_mask): $[x_t(B1-B3) \parallel x_0(B1'-B3')]$ 입력에서 노이즈 블록 B3의 쿼리는 자기 블록 (BD)과 이전 블록들의 클린 x_0 (OBC — 이전 블록 정보의 유일한 학습 동료)만 본다. BC는 클린 절반의 표현 유지용.

오른쪽 (샘플링, get_block_causal_mask): 현재 블록 B3*의 모든 토큰이 커밋된 B1, B2를 매 레이어에서 직접 attend (주황 강조). 커밋 토큰은 동결($t=0$)이며 KV 캐시 없이 매 forward 재인 코딩.

adaLN 조건 경로($c = t + OD$ 임베딩)는 이전 블록의 내용을 나쁘지 않는다 — 내용은 100% attention으로 흐른다.

C. BDTransformer 타워 — 아래에서 위로 읽기; 조건과 이전 블록 내용의 정확한 주입 지점



- γ, β, α 는 모델 파라미터가 아니다. 학습 파라미터는 보라색 adaLN MLP의 가중치(zero-init → 초기엔 전 블록 항등)이고, $\gamma/\beta/\alpha$ 는 그 MLP가 토큰-샘플마다 c 에서 계산하는 활성값이다. 조건은 가중치를 바꾸지 않고 정규화된 활성값을 feature별 scale($1+\gamma$)/shift(β)하고 서브레이어 출력을 gate(α)한다 — LN이 affine-free인 이유.
- 이전 블록의 내용은 adaLN이 아니라 attention의 K/V로 들어온다 (파란 박스). 요약: 내용은 K/V로, 조건(블록별 t-OD)은 $\gamma\beta\alpha$ 로.

```
(gamma_1, beta_1, alpha_1, gamma_2, beta_2, alpha_2) = adaLN_MLP(c).chunk(6) # BDBlock.forward, bd_models.py:149-153
x = x + alpha_1 * Attn((1+gamma_1) * LN(x) + beta_1, attn_mask)
x = x + alpha_2 * MLP((1+gamma_2) * LN(x) + beta_2)
```

D. Open-ended semi-AR 생성: 블록 append → first-hitting으로 토큰 1개씩 공개(공개 토큰은 즉시 다음 forward의 문맥) → 커밋 블록 스택: dst → hit(정지) / END → miss(정지) / 아니면 다음 노이즈 블록 append.

학습의 OBC(노이즈→클린 과거)와 샘플링의 커밋-참조가 정확히 대응하도록 설계되어 (teacher forcing), 학습-생성의 조건화 분포가 일치한다. blk1에서는 모든 정보가 커밋-참조로만 흘러 causal LM이 되고, blk64(=MDLM)에서는 OBC가 공집합이 된다 — block size가 두 통로의 비율을 보간한다 (§6.9).

2. Block size sweep — held-out, 시드 3개 mean±std

후처리 상태: 없음. §2(\$2.1-§2.2-§2.3)의 모든 지표는 RAW 모델 출력이다 — P1/P3-refine 일절 없음. 후처리는 §3부터만 등장한다.

2.1 mask

blk	hit (=arr)	valid	valid ^ hit (raw)	EM	PC	DTW	len err	len corr
1	0.997±.002	0.934±.007	0.934±.007	0.830±.011	0.869±.010	0.032	0.84	+0.864
2	0.958±.007	0.842±.006	0.822±.006	0.769±.005	0.799±.005	0.038	0.45	+0.982
4	0.958±.001	0.836±.004	0.811±.004	0.747±.010	0.783±.008	0.037	0.37	+0.991

8	0.951±.002	0.808±.006	0.774±.006	0.716±.008	0.754±.007	0.042	0.40	+0.993
16	0.898±.006	0.680±.006	0.625±.002	0.607±.009	0.637±.008	0.061	0.56	+0.989
32	0.858±.010	0.474±.004	0.420±.002	0.413±.002	0.437±.002	0.081	0.96	+0.968
64	0.839±.005	0.430±.014	0.373±.010	0.366±.012	0.391±.012	0.072	0.89	+0.988

2.2 graph (d = 2.0)

blk	hit (=arr)	valid	valid∧hit (raw)	EM	PC	DTW	len err	len corr
1	0.429±.001	1.000±.000	0.429±.001	0.093±.006	0.168±.007	0.437	36.4	+0.237
2	0.173±.014	0.997±.001	0.173±.014	0.399±.008	0.558±.005	0.488	6.48	+0.492
4	0.128±.007	0.978±.002	0.128±.007	0.309±.012	0.499±.008	0.302	4.50	+0.719
8	0.058±.004	0.991±.001	0.058±.004	0.124±.012	0.331±.006	0.282	4.53	+0.783
16	0.029±.002	0.996±.002	0.029±.002	0.148±.008	0.390±.006	0.377	5.66	+0.705
32	0.035±.002	0.969±.002	0.035±.002	0.174±.002	0.411±.004	0.394	5.96	+0.693
64	0.027±.004	0.966±.003	0.027±.004	0.198±.006	0.430±.001	0.363	5.74	+0.724

2.3 실패 해부 (v3, held-out 1,000쌍, 생성 시드 7 고정, RAW)

후처리 상태: 없음. §2.1/2.2와 동일 설정(양 커널-전 7개 블록 크기)을 동일한 held-out 1,000쌍에서, 단일 고정 시드(시드 평균 없음)로 raw 측정 한 것. "경계/내부" = invalid edge가 블록의 첫 토큰 (이전 블록에서 예측, AR형)에 놓였는지 vs 블록 내부 토큰에 놓였는지.

mask	hit	len corr (hit)	len corr (miss, n)	miss→dst 중앙	miss→dst 평균	≤2 hops	inv-edge	경계/내부
1	0.997	0.972	- (3)	14	13.0	0%	1.02%	234 / 0
2	0.956	0.989	0.962 (44)	2	2.41	86%	1.70%	192 / 189
4	0.954	0.987	0.976 (46)	2	1.93	87%	1.03%	35 / 196
8	0.954	0.995	0.974 (46)	2	3.22	89%	1.06%	25 / 211
16	0.909	0.992	0.959 (91)	2	3.32	89%	1.98%	12 / 429
32	0.871	0.963	0.905 (129)	2	2.95	84%	4.27%	7 / 920
64	0.853	0.991	0.974 (147)	2	3.67	84%	4.54%	0 / 1001

graph d=2.0	hit	len corr (hit)	len corr (miss, n)	miss→dst 중앙	miss→dst 평균	≤2 hops	inv-edge
1	0.432	0.458	0.054 (568)	9	13.5	5%	0.00%
2	0.194	0.902	0.463 (806)	4	10.7	31%	0.00%
4	0.119	0.969	0.723 (881)	4	6.93	34%	0.00%
8	0.072	0.978	0.759 (928)	5	7.82	26%	0.00%
16	0.032	0.996	0.702 (968)	5	8.40	19%	0.00%
32	0.029	0.997	0.703 (971)	6	9.39	17%	0.00%
64	0.027	0.994	0.729 (973)	6	8.96	17%	0.00%

- **mask의 miss는 near-miss:** blk2-64에서 miss 종착점이 dst에서 중앙값 2 hop (84-89%가 ≤2) — 마지막 토큰 식별 실패라 P3 endpoint patch로 저렴하게 복구됨. 유일한 예외는 blk1: miss는 3개뿐이지만 멀리 벗어남(중앙값 14 hop, ≤2 0%) — 드문 순수-AR 폭주. 길이 보정은 hit(0.96-1.00)·miss(0.90-0.98) 전 구간 유지.
- **validity 하락 = 블록 내부 스킵, 전 스킵에서 단조 관찰:** 블록이 커질수록 invalid edge가 경계→내부로 이동 — blk2 192/189 → blk4 35/196 → blk16 12/429 → blk64 0/1001(전부 내부). blk1은 정의상 전부 경계(모든 토큰이 블록 시작). 블록 경계(=AR형) 전이는 무결하고, 비용은 **블록 내부**에서만 발생 — first-hitting의 블록 내 무작위 공개 순서가 원인. edge 오류율은 1.0%→4.5%로만 오르지만(중앙값 2-3-hop 스킵), 경로 단위 all-or-nothing이 이를 증폭한다.
- **graph 커널은 거울상 — 그리고 그 miss는 near-miss가 아니다:** 제약 디코드로 inv-edge가 전 블록에서 0.00%이지만, 블록이 커질수록 hit이 0.43→0.03으로 붕괴. mask와 달리 graph의 miss 종착점은 dst에서 멀리 놓인다(중앙값 4-9 hop, 평균 7-13.5, ≤2가 5-34%뿐) — 평균이 중앙값보다 훨씬 크다는 것은 오른쪽 꼬리가 두껍다는 뜻(일부 miss가 아주 멀리 이탈)이고, 반면 mask miss는 평균≈중앙값≈2로 꼬리가 없다. 마지막 토큰 실수가 아니라 진짜 내비게이션 이탈이라 endpoint patch로 저렴하게 복구 불가. 커널별 비용: mask는 validity로, graph는 arrival로 지름.
- **held-out 검증:** v2 수치가 v1(train 중첩 가능)과 노이즈 이내 일치 — 암기 인플레이션 없음. 경로 193만 대비 파라미터 6.8M(경로당 ~3.5개)라 축약적 암기가 불가능하고, 모델이 배우는 것 — 고정된 목적지를 향한 국소 전이 선택 — 은 미지의 OD쌍에 그대로 전이된다.
- **미실험 노트:** END tail의 loss 비중이 (B-1)/2로 커져 blk64에서 감쪽 토큰의 ~58%가 END — 상수-질량 재가중/첫-END-만 등 loss engineering 여지 (v1 §4.5).

3. 후처리: P1 (local splice) · P3 (endpoint patch)

P1: invalid 쌍 (u,v)마다 최단경로 삽입 → valid = 1 보장, hit 불변. P3(refine_nx): miss 중점에서 dst까지 graft → arrival = 1 보장. hit은 항상 후처리 전 측정(valid ∧ hit (raw)) — P1+P3 하에서 남은 유일한 모델 의존량.

blk	raw arr	raw valid	valid ∧ hit (raw)	raw EM	raw PC	P1+P3 EM	P1+P3 PC	패치
1	0.997	0.934	0.934	0.832	0.871	0.849	0.898	1.0
2	0.956	0.852	0.826	0.779	0.810	0.804	0.869	0.5
4	0.954	0.823	0.798	0.734	0.771	0.768	0.846	0.8
8	0.954	0.827	0.794	0.728	0.769	0.768	0.848	0.7
16	0.909	0.705	0.647	0.617	0.651	0.662	0.766	1.9
32	0.871	0.462	0.408	0.397	0.422	0.503	0.642	3.3
64	0.853	0.441	0.390	0.379	0.403	0.533	0.661	3.5

- P1+P3는 모든 블록에서 valid = arrival = 1의 구조적 보장을 주고, 사용 가능 경로 비율 = raw hit. 큰 블록에서 P1이 EM/PC도 올린다(blk64 EM 0.379→0.533) — 브리지가 실제 경로 조각이라서. 수선 불가 쌍 0건.

4. Importance-weight guidance (예외 상황 except_0)

수식 요약 (전문: BD_GUIDANCE_FORMULATION.pdf). 블록 b의 guided reverse: $q(x_{t-1}^b | x_t^b, x^{<b}) = \mathbb{E}_{x \sim p} [w \cdot q(\cdot | x_t)] / \mathbb{E}[w]$, $w = q(x_0 | x^{<b}) / p_\theta(x_0 | x^{<b}) \approx D / (1-D)$. **Lemma 1:** $w = C^{-1} \cdot r(x^{<b} \| x_0)$, $C = q(x^{<b}) / p_\theta(x^{<b})$ 는 후보-독립 상수라 자기정규화에서 소거 — **클린 부분-경로** discriminator로 충분. **Lemma 2:** 블록만 보는 분류기는 marginal ratio를 배워 편향(상수 아님 → 미소거). **Lemma 3:** 후보 proposal을 공개-이웃 기준 A_{scn} -합법 전이로 마스킹해도 정확 — 제외 후보는 $\text{supp}(q)$ 밖이라 $w=0$ 인 항뿐이고 Z_t 는 후보-공통이라 소거 (비편향 분산 감소). 구현: first-hitting reveal마다 mean-field 후보 $N=100$, $w=\text{exp}(\text{bounded logit})$, 매 reveal 적용, ESS 로깅. Discriminator: GraphSAGE+edge-existence 채널(시나리오 인접행렬 입력 → 재학습 없는 시나리오 전이), negative = 해당 블록 모델의 unconditional 생성 20k($E(q)$ 의 분모 p_θ 그대로; data<mix<model 순열은 v1 §6.7).

4.1 Seen (except_0으로 disc 학습, 1%)

후처리: "PIP3" 태그가 없는 컬럼(base/IW/adj+IW valid EM, rem-edge, ESS)은 RAW이고, 명시된 P1P3 EM/PC 컬럼만 P1+P3 적용.

blk	base valid	base EM	IW valid	IW EM	IW P1P3 EM	IW P1P3 PC	ESS	adj+IW valid	adj+IW EM	rem-edge	adj+IW P1P3 EM	adj+IW P1P3 PC
4	0.266	0.240	0.331	0.298	0.309	0.464	99.1	0.390	0.338	3.0%	0.357	0.496
8	0.231	0.212	0.298	0.264	0.294	0.453	98.8	0.346	0.298	3.2%	0.335	0.483
16	0.222	0.198	0.277	0.240	0.268	0.423	98.2	0.327	0.270	3.1%	0.295	0.447
32	0.160	0.132	0.225	0.184	0.216	0.376	97.5	0.255	0.203	3.2%	0.236	0.398
64	0.141	0.129	0.201	0.160	0.211	0.372	95.4	0.229	0.176	3.1%	0.230	0.394

4.1b Negative 종류: 모델 생성 vs normal 데이터 (disc 구조 동일, e0, raw)

위 §4.1 수치는 **model-negative** disc(negative = 그 블록의 unconditional 생성 → 밀도비 $q_{\text{except}}/p_\theta$, 이론적으로 올바른 분모)를 쓴다. 여기서는 동일 프로토콜을 **data-negative** disc(negative = normal 실제 경로 → $q_{\text{except}}/q_{\text{normal}}$, 즉 시나리오-이동 비율이지 q/p_θ 가 아님)로 다시 돌린다. P1P3 열만 후처리, 나머지 raw; 같은 v2 백본·같은 except_0 예약 1,000쌍·seed 7.

blk	base valid	IW valid (model / data)	adj+IW valid (model / data)	adj+IW EM (model / data)	adj+IW rem-edge (model / data)	adj+IW P1P3 EM (model / data)
4	0.266	0.331 / 0.320	0.390 / 0.375	0.338 / 0.333	3.0% / 2.9%	0.357 / 0.349
8	0.231	0.298 / 0.285	0.346 / 0.332	0.298 / 0.288	3.2% / 3.2%	0.335 / 0.334
16	0.222	0.277 / 0.258	0.327 / 0.308	0.270 / 0.257	3.1% / 3.1%	0.295 / 0.294
32	0.160	0.225 / 0.194	0.255 / 0.235	0.203 / 0.183	3.2% / 3.2%	0.236 / 0.227
64	0.141	0.201 / 0.182	0.229 / 0.211	0.176 / 0.168	3.1% / 3.1%	0.230 / 0.225

- 둘 다 실제 이득을 주며, model-negative가 전 구간에서 (근소하게) 우세: base 대비 blk4에서 data-negative IW는 valid +5.4%p(0.320 vs 0.266), model-negative는 +6.5%p(0.331). 이 격차는 모든 블록 크기와 adj+IW에서도 유지(blk64: model 0.229 vs data 0.211). v2 백본에서 data<model 순서를 실증.
- model-negative가 이론적으로 옳은 이유: guidance는 $w = q_{\text{except}}(x_0 | x^{<b}) / p_\theta(x_0 | x^{<b})$ 를 필요로 한다. model-negative는 disc가 바로 이 비율(분모 = p_θ)을 학습하게 한다. data-negative는 $q_{\text{except}}/q_{\text{normal}}$ 를 학습 — 방향("normal 전용 경로 회피")은 맞지만 분모가 틀려 tilt가 약간 어긋난다.
- rem-edge는 negative 종류와 무관: adj+IW의 제거-edge 사용률은 둘 다 ~3%(2.9-3.2%) — 이 감소는 disc가 아니라 Lemma 3 인접성 마스킹에서 오기 때문.

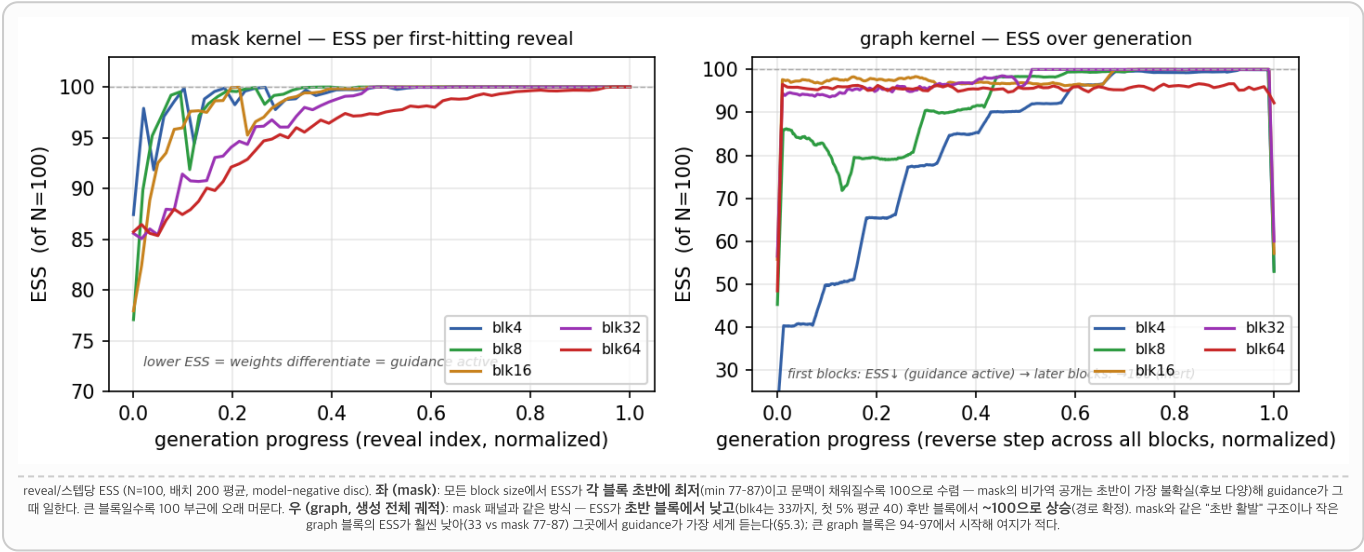
4.2 Unseen (except 1-99로 disc 학습 → except_0 평가)

blk	IW valid (unseen/seen)	adj+IW valid (unseen/seen)	adj+IW PIP3 EM (unseen/seen)
4	0.325 / 0.331	0.382 / 0.390	0.348 / 0.357
16	0.270 / 0.277	0.321 / 0.327	0.299 / 0.295

- **분업**: 제거 edge 회피는 Lemma 3 마스킹이(rem-edge ~5%→3%), 경로 품질은 disc 재가중이(+5~7%p valid) 담당. ESS ≈ 95-99: 스텝당 개입은 작지만 reveal 전반에 누적. 비용은 arrival ~5~7%p — P3가 구조적으로 복원.
- **일반화 격차 ≈ 0** (blk4: 0.382 vs 0.390): 시나리오 정보가 암기가 아니라 인접행렬 입력으로 들어간다는 증거. 완성형(adj+IW+PIP3)은 blk4에서 EM 0.357/PC 0.496 — baseline+PIP3(0.270/0.431) 대비 상대 +32%/+15%.

4.3 ESS 궤적 — block sizes 4-64

ESS = $(\sum w)^2 / \sum w^2 \in [1, N=100]$ 읽는 법: 값이 클수록(→100) 통계적으로 많은 후보가 살아있다는 뜻이지만, importance-weight guidance에서는 그것이 **가중치가 균등 = disc가 후보를 구분 못 함 = guidance 무력**을 의미한다. 값이 낮을수록(예: 33) 소수 후보에 가중치 집중 = **disc가 강하게 선별 = guidance 작동**. 통계적 "유효 표본 수"와 guidance 관점의 "살모"는 방향이 반대다.



5. 발표 baseline과의 통제 비교: D-CBG (Schiff et al.)

Plug-in 아식(그들 코드 로직 그대로: dcbg_plugin.py), 분류기 아키텍처·학습 프로토콜 완전 일치(같은 GraphSAGE+edge 채널, 같은 1% 데이터, model-negative는 같은 v2 uncond pool을 그들 방식대로 노이즈). 남은 차이는 방법 내재적인 것뿐: 노이즈-상태 입력 + 위치별 tilting (D-CBG) vs 클린 부분-경로 + joint 후보 재가중 (IW). $\gamma=4$ ($\gamma \leq 2$ 는 무력, v1 §7.1). exact 열거는 reveal당 $|V| \approx 1.4k$ 분류기 호출(≈ 2.7 s/path), first-order 근사는 저렴하나(0.07 s/path) **v2에서도 무력 재확인**(blk4 valid 0.265 $\approx$ base 0.266); IW는 2.4k 호출/0.03 s/path.

열 정의(명시): D-CBG(data/model) = $\gamma=4$ 의 exact 열거(1차 근사 아님; 근사 버전은 아래 별도 행) + adjacency-aware 분류기, negative는 각각 normal 실패데이터 / v2 uncond pool. **IW(model) = model-negative**(같은 v2 pool)로 학습한 부분-경로 disc의 importance-weight 재가중. **adj+IW = IW(model)과 완전히 같은 disc**에 Lemma 3 proposal 마스킹만 추가한 것 (즉 네, model-data 학습 disc가 맞음). **컬럼별 후처리**: 모든 *valid* 컬럼은 RAW(후처리 전), 모든 *PIP3 EM* 컬럼은 P1+P3 적용 후 — 각 방법의 raw validity와 후처리 EM이 나란히 있다.

blk	D-CBG(data) valid	PIP3 EM	D-CBG(model) valid	PIP3 EM	IW(model) valid	PIP3 EM	adj+IW valid	PIP3 EM
4	0.301	0.295	0.309	0.302	0.331	0.309	0.390	0.357
8	0.257	0.275	0.272	0.275	0.298	0.294	0.346	0.335
16	0.249	0.263	0.258	0.252	0.277	0.268	0.327	0.295
32	0.185	0.231	0.185	0.203	0.225	0.216	0.255	0.236
64	0.163	0.222	0.173	0.209	0.201	0.211	0.229	0.230

D-CBG 1차 근사(first-order/Taylor), 전 블록 실측 ($\gamma=4$, model-negative 분류기): blk4: 0.265 (base 0.266) · blk8: 0.233 (base 0.231) · blk16: 0.222 (base 0.222) · blk32: 0.161 (base 0.160) · blk64: 0.143 (base 0.141) — **모든 블록에서 무력** (전부 base와 노이즈 이내). one-hot MASK 임베딩에서 취한 gradient가 치환 효과를 근사하지 못한다. wall-clock 0.074-0.090 s/path.

5.1 Wall-clock 시간 (정의는 §1.2)

방법	blk4 s/path	blk64 s/path	비고
base (guidance 없음)	0.009	0.007	—
IW (model)	0.026	0.066	reveal당 disc 호출 ~100 (미니배치 3회)
adj+IW	0.026	0.067	마스킹 비용은 무시 가능

D-CBG exact (y=4)	0.48-0.57	1.00-1.08	reveal당 분류기 호출 V ≈1,393
D-CBG 1차 근사	0.074-0.27	—	reveal당 fwd+bwd 1회이나 무력

측정: 유티 RTX 3090 1장, 배치 100. IW/base는 200쌍 타이밍 루프(생성만, CUDA sync 포함), D-CBG exact는 1,000쌍 v3 본 실행과 200쌍 재측정의 범위, 1차 근사는 1,000쌍(0.074)과 200쌍 (0.27, 시작 오버헤드 포함)의 범위. 해석: IW는 exact D-CBG보다 blk4에서 ~20배, blk64에서 ~15배 빠르다 — 호출 수 차이(2.4k vs 33k)에 더해, IW의 호출은 인덱스 입력 forward뿐이라 마이크 로배칭 효율이 높기 때문. 1차 근사는 빠르지만 효과가 없으므로 비용-품질 프론티어에서 지배당한다.

5.2 치팅 감사 — IW의 우위는 어디서 오는가

통제된 것: 평가셋(같은 except_0 예약 1,000쌍-같은 시드), 분류기 아키텍처(GraphSAGE+edge 채널·bounded logit-label smoothing 동일), 학습 데이터(같은 1%-같은 split-같은 negative pool), 백본(같은 v2 체크포인트). D-CBG에는 y∈{1,2,4} sweep 후 최적(y=4)을 보고하므로 튜닝은 오히려 D-CBG에 유리. D-CBG exact는 reveal당 |V|≈1,393회 평가로 IW(100회)보다 더 많은 분류기 정보를 사용한다.

blk	base	placebo-IW (균등 가중)	IW (진짜 disc)	placebo-adj (마스킹만)	adj+IW (진짜)
4	0.266	0.267	0.331	0.331	0.390
16	0.222	0.211	0.277	0.281	0.327
64	0.141	0.143	0.201	0.179	0.229

- Placebo 대조군 (ESS = 100.0으로 강제): 무학습 상수-logit disc로 IW 배관 전체(후보 100개 샘플링 → $\bar{x}$ 평균 → reveal)를 그대로 돌리면 base와 노이즈 이내로 일치 (blk4 0.267 vs 0.266). 즉 IW의 이득은 후보 평균화·샘플링 배관·구현 부산물이 아니라 전적으로 discriminator 신호에서 온다. placebo-adj는 마스킹 단독 효과(+6~7%p)를 분리해 주며, 진짜 disc가 그 위에 다시 +5~6%p를 얻는다 — §4의 분업 분해가 인과적으로 확인됨.
- 남는 유일한 비대칭은 방법 내재적이다: D-CBG의 수식은 노이즈 상태 $p(y|x,t,t)$ 분류를 요구하고, IW의 수식(Lemma 1)은 클린 부분-경로 ratio로 충분하다. 같은 아키텍처·데이터에서 분류기 품질은 비슷하게 학습되지만(val 0.73-0.75/0.82-0.83 동급), 노이즈-입력 분류가 uniform 커널에서는 아예 학습 불가(5.3)라는 구조적 차이가 남는다. 이것은 치팅이 아니라 정식화의 차이 — 그리고 그것이 바로 방법론의 기여다.
- 누수 점검: diffusion은 예외 데이터를 본 적 없음, disc/분류기는 평가 1,000쌍 제외 후 학습, 평가쌍은 두 방법이 동일. y 튜닝·분류기 호출 수는 D-CBG에 유리하게 설정됨.

5.3 GDP graph 커널에서의 비교 (v2 graph blk4/64)

blk	방법	valid raw	EM raw	arrival	ESS	PIP3 EM
4	base	0.393	0.138	0.137	—	—
	IW (model)	0.478	0.184	0.115	84.3	0.130
	adj+IW	0.971	0.334	0.042	81.5	0.182
	D-CBG fo (data)	0.404	0.134	0.114	—	0.086
	D-CBG fo (model)	0.417	0.127	0.132	—	0.085
8	base	0.427	0.069	0.064	—	—
	IW (model)	0.568	0.150	0.040	92.0	0.115
	adj+IW	0.955	0.215	0.015	90.1	0.134
	D-CBG fo (data)	0.419	0.067	0.044	—	0.046
	D-CBG fo (model)	0.459	0.084	0.045	—	0.047
16	base	0.498	0.114	0.030	—	—
	IW (model)	0.678	0.177	0.010	98.0	0.115
	adj+IW	0.980	0.237	0.002	97.7	0.133
	D-CBG fo (data)	0.505	0.111	0.023	—	0.068
	D-CBG fo (model)	0.496	0.102	0.017	—	0.065
32	base	0.462	0.089	0.029	—	—
	IW (model)	0.660	0.146	0.015	97.7	0.127
	adj+IW	0.930	0.225	0.004	96.3	0.164
	D-CBG fo (data)	0.449	0.084	0.024	—	0.070
	D-CBG fo (model)	0.467	0.091	0.027	—	0.069
64	base	0.473	0.108	0.034	—	—
	IW (model)	0.684	0.189	0.013	95.8	0.125
	adj+IW	0.962	0.248	0.008	95.8	0.147

	D-CBG fo (data)	0.465	0.122	0.042	—	0.085
	D-CBG fo (model)	0.469	0.121	0.036	—	0.082

- 가설 확인: uniform limiting distribution의 graph 커널에서 guidance 효과가 훨씬 크다. IW만으로 valid +8.5%p(blk4)/+21%p(blk64), adj+IW는 0.97/0.96으로 validity를 사실상 해결 — mask 커널의 +6.5%p와 대비. ESS는 커널 간 차이(graph > mask)를 설명한다: graph에서는 가중치가 실제로 같린다 — 첫 블록 최저 ESS가 33(blk4)~94(blk16)로 mask의 ~77-87(곧 100 복귀)보다 낮아 uniform proposal이 disc에 고를 후보를 준다. 그러나 graph 내부의 블록 사이즈 경향은 ESS와 정반대다: IW validity 이득은 블록이 클수록 커지는데(+8.5%p@blk4 → +21%p@blk64) ESS는 blk4에서 가장 낮다. ESS는 “지금 disc가 후보를 구분하나”의 순간 지표라 역스텝 수·base 여유를 적분하지 않으므로 graph-vs-mask는 설명해도 블록 순서는 설명하지 못한다. blk4에서 raw EM 0.334, PC 0.512는 이 벤치마크 graph 커널의 최고 기록. 전 블록(4-64) 확장 확인: adj+IW valid가 모든 graph 블록에서 0.93-0.98로 validity를 사실상 해결하고 (IW 단독 0.48-0.68), D-CBG first-order는 두 negative 모두 전 블록에서 무력(≈base) — blk4/64 패턴이 그대로 유지된다.
- D-CBG는 graph에서 실행 가능한 변형이 없다: exact는 $T \times \text{위치} \times |V| \approx 890$ 만 호출로 계산 불가, 1차 근사는 두 negative 모두에서 무력(≈ base). 노이즈-상태 분류기도 data-negative에서는 우연 수준(val 0.53-0.54; model-negative는 0.87로 학습되나 근사가 신호를 전달 못 함).
- 비용: graph IW ≈ 0.4-0.9 s/path (T=100 reverse × 후보 채점), D-CBG fo ≈ 0.5 s/path — graph에서는 비용이 비슷하나 효과는 IW만 있다.
- 모든 블록에서 IW > D-CBG (raw valid 10/10; blk4 0.331 vs 0.309), 완성형 adj+IW가 전 셀 최고. 우리 재료 중 시나리오-조건 분류기와 model-negative는 D-CBG에도 이식되어 그들을 개선 하지만, 순서는 유지된다.
- Unseen 전이도 양쪽 성립, IW 우위 유지: blk4 D-CBG 0.306 / IW 0.325 (seen 0.309 / 0.331). 일반화는 인접행렬-조건 분류기 설계(우리 기여, 이식 가능)의 속성.
- 비용-품질 프론티어: 싼 D-CBG(first-order)는 작동하지 않고, 작동하는 D-CBG(exact)는 싸지 않다 — IW가 그 사이의 지배점 (30-90× 저렴, γ 튜닝 불필요, arrival 보존). uniform 커널에서는 그들의 노이즈-상태 분류기가 구조적으로 학습 불가(우연 수준)라 격차가 결정적 (v1 §7).

6. 결론과 산출물

- mask 커널의 작은 블록(1-8)이 과제를 사실상 해결하고, 남은 결함(국소 스킵·near-miss)은 P1/P3가 구조적으로 보정한다. graph 커널은 validity-arrival 트레이드의 반대극.
- 예외 상황 대응의 완성형: adj-masked proposal + model-negative partial-path disc(IW) + P1/P3 — seen/unseen 모두에서 최고이며, 발표 baseline(D-CBG)을 모든 matched 구성에서 상회한다.
- 모델: sets_model/*_v2_bd.pth (147개; v1은 v1_archive/). 결과: sets_res/, 로그: sets_log/v3_*. 코드: bd_models.py(plan_guided), bd_disc.py, dcbg_plugin.py, train_bd_disc.py, train_dcbg_classifier.py, eval_*.py, run_v3_all.sh.